

AFRILANG Stdlib

std/m/listmerge2

Bibliothèque AFRILANG `std/m/listmerge2` (niveau medium, catégorie medium). Elle expose 4 fonction(s) : mergeSorted, mer

```
`import "std/m/listmerge2" . `use listmerge2`
```

- `mergeSorted(a list number, b list number) ? list number`
- `mergeAll(a list number, b list number) ? list number`
- `intersect(a list number, b list number) ? list number`
- `unionAll(a list number, b list number) ? list number`

Category: medium | Tier: medium | Functions: 4

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG ``std/m/listmerge2`` (niveau medium, catégorie medium). Elle expose 4 fonction(s) : `mergeSorted`, `mergeAll`, `intersect`, `unionAll`

```
`import "std/m/listmerge2"` . `use listmerge2`  
- `mergeSorted(a list number, b list number) ? list number`  
- `mergeAll(a list number, b list number) ? list number`  
- `intersect(a list number, b list number) ? list number`  
- `unionAll(a list number, b list number) ? list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/m/listmerge2"  
use listmerge2
```

Niveau : medium · Catégorie : Modules medium (m/)

Bibliothèques intermédiaires ? import std/m/?

3. Référence API

```
? mergeSorted(a list number, b list number) ? list  
? mergeAll(a list number, b list number) ? list  
? intersect(a list number, b list number) ? list  
? unionAll(a list number, b list number) ? list
```

4. Exemples d'utilisation

```
import "std/m/listmerge2"  
use listmerge2  
  
create result = mergeSorted(/* args */)   
say result  
  
create result = mergeAll(/* args */)   
say result  
  
create result = intersect(/* args */)   
say result  
  
create result = unionAll(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez ``import "std/m/listmerge2"`` en tête de fichier.
- ? Lancez ``afrilang check`` avant de livrer.
- ? Combinez avec les modules `stdlib` de la même catégorie.
- ? Voir `/stdlib/` pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module listmerge2  
  
function mergeSorted(a list number, b list number) returns list number  
end  
  
function mergeAll(a list number, b list number) returns list number  
end  
  
function intersect(a list number, b list number) returns list number  
end  
  
function unionAll(a list number, b list number) returns list number  
end  
end
```

ENGLISH

1. Overview

AFRILANG library `std/m/listmerge2` (tier medium, category medium). Exports 4 function(s): mergeSorted, mergeAll, inters

```
`import "std/m/listmerge2"` . `use listmerge2`  
- `mergeSorted(a list number, b list number) ? list number`  
- `mergeAll(a list number, b list number) ? list number`  
- `intersect(a list number, b list number) ? list number`  
- `unionAll(a list number, b list number) ? list number`
```

2. Installation & import

Add the module to your program:

```
import "std/m/listmerge2"  
use listmerge2
```

Tier: medium · Category: Medium modules (m/)
Intermediate libraries ? import std/m/?

3. API reference

```
? mergeSorted(a list number, b list number) ? list  
? mergeAll(a list number, b list number) ? list  
? intersect(a list number, b list number) ? list  
? unionAll(a list number, b list number) ? list
```

4. Usage examples

```
import "std/m/listmerge2"  
use listmerge2  
  
create result = mergeSorted(/* args */)   
say result  
  
create result = mergeAll(/* args */)   
say result  
  
create result = intersect(/* args */)   
say result  
  
create result = unionAll(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/m/listmerge2"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module listmerge2  
  
function mergeSorted(a list number, b list number) returns list number  
end  
  
function mergeAll(a list number, b list number) returns list number  
end  
  
function intersect(a list number, b list number) returns list number  
end  
  
function unionAll(a list number, b list number) returns list number  
end  
end
```


